

NUCLEO-G474RE

Power Electronics Project Series

Project Documentation

05

H-Bridge Motor Control with HRTIM Complementary PWM

Implementation Frameworks:

STM32CubeIDE | MATLAB Embedded Coder

Document Version: 1.0

Last Updated: April 16, 2026

GitHub: <https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>

This work is licensed under the MIT License

Contents

1	Project Overview	3
1.1	Objective	3
1.2	Applications	3
1.3	Learning Outcomes	3
2	STM32CubeIDE Implementation	4
2.1	Prerequisites	4
2.2	Hardware Configuration	4
2.2.1	Pin Configuration	4
2.3	Peripheral Configuration	5
2.4	Software Architecture	10
2.4.1	Project Structure	10
2.4.2	Initialisation Flow	11
2.5	Control Logic and FSM	11
2.6	Code Implementation	12
2.6.1	Key Defines and Data Types	12
2.6.2	Main Initialisation	13
2.6.3	Core Functionality	13
2.6.4	HRTIM Output Helpers	15
2.6.5	Interrupt Service Routines	15
2.7	Debugging	16
3	MATLAB Embedded Coder Implementation	17
3.1	Prerequisites	17
3.2	MATLAB/Simulink Model Design	17
3.2.1	Model Architecture	17
3.2.2	Block Functionality	18
3.3	Model Configuration	18
3.3.1	CubeMX Configuration	18
4	STM32CubeIDE vs. MATLAB Embedded Coder	23
4.1	Implementation Comparison	23
4.2	Key Differences in this Project	23
4.3	Hybrid Approach Recommendation	24
5	Testing and Validation	24
5.1	Visual Verification	24
5.2	Functional Testing	24
5.3	Performance Characteristics	24
6	Results and Analysis	24
6.1	Expected Outcomes	24

6.2	Key Observations	25
6.3	Project Validation	25
7	Troubleshooting	25
7.1	Common Issues and Solutions	26
7.2	Debugging Recommendations	26
8	Extensions and Enhancements	26
8.1	Further Learning	26
9	Conclusion	27
9.1	Key Takeaways	27
9.2	Next Steps	27
A	HRTIM PWM Configuration Register Summary	29
B	ADC1 Configuration Summary	30
C	Full Source Code Listings	30
D	Hardware Documentation	31

1 Project Overview

1.1 Objective

This project implements a closed-loop, variable-speed H-bridge motor controller on the STM32 NUCLEO-G474RE using the on-chip High-Resolution Timer (HRTIM). The controller generates asymmetric complementary PWM at 40 kHz with hardware-enforced 1 μ s dead-time on both switching edges. An analogue potentiometer connected to ADC1 Channel 1 provides a real-time duty-cycle reference in the range 0 to 1. A master ramp integrator ensures graceful acceleration and deceleration, protecting the motor and the H-bridge switches from abrupt current transients.

The system integrates three distinct operational states managed by dedicated GPIO interrupt lines: forward drive (Direction A), reverse drive (Direction B), and a controlled ramp-down stop sequence. This project represents the culmination of the fundamentals introduced in earlier modules—GPIO interrupts, ADC acquisition, and HRTIM configuration—unified into a complete, safety-conscious power electronic drive.

1.2 Applications

H-bridge converters are a foundational topology in power electronics. Specific application areas include:

- **DC Motor Drives:** Bidirectional speed and torque control of brushed DC motors in robotics, electric vehicles, and industrial automation.
- **Synchronous Buck/Boost Converters:** The complementary switching pattern directly maps to non-inverting buck-boost topologies.
- **Class-D Audio Amplifiers:** High-efficiency audio output stages exploit the same MOS-FET half-bridge switching principle.
- **Battery Chargers and BMS:** Bidirectional DC-DC stages requiring precise current control and safe transition management.
- **Actuator Control:** Linear actuators, solenoid valves, and stepper motor half-bridges all share the H-bridge switching topology.

Dead-time insertion is critical in all of these applications. Without it, simultaneous conduction of high-side and low-side switches (shoot-through) creates a direct short across the supply, destroying the power devices. The HRTIM's hardware dead-time mechanism makes this protection deterministic and independent of software execution timing.

1.3 Learning Outcomes

Upon completing this project the reader will be able to:

- Configure the STM32G474RE HRTIM with a $\times 32$ DLL multiplier for 680 MHz resolution timing.

- Derive period and dead-time tick values from first principles and verify them against HAL register writes.
- Implement asymmetric complementary PWM using the four compare units (CMP1–CMP3, CMP2) of Timer A.
- Design a three-state finite-state machine (FSM) controlled entirely from GPIO EXTI callbacks with ISR-safe flag passing.
- Apply a discrete-time ramp integrator in firmware to perform safe deceleration and prevent back-EMF spikes.
- Replicate the same control architecture in MATLAB/Simulink using Multiplexers, Discrete-Time Integrators, and Saturation blocks, then deploy via Embedded Coder.
- Appreciate the role of safety clamps and guard conditions in production power electronic firmware.

2 STM32CubeIDE Implementation

2.1 Prerequisites

Before proceeding, ensure you have explored all prior modules in the series—each one builds a targeted skill (GPIO, ADC, HRTIM basics) that this project relies upon simultaneously. The complete repository, including all prerequisite projects, schematic references, and MATLAB models, is available at:

<https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>

2.2 Hardware Configuration

2.2.1 Pin Configuration

Table 1: GPIO and Peripheral Pin Assignments

Pin	Signal	Mode	Function
PA8	HRTIM_CHA1 (TA1)	Alternate Function	High-side PWM output
PA9	HRTIM_CHA2 (TA2)	Alternate Function	Low-side complementary PWM output
PA1	ADC1_IN1	Analogue Input	Duty-cycle reference (potentiometer)
PC13	B1_Pin	EXTI Rising Edge	Stop / Resume button (no pull, board pull-down)
PB15	Direction Button	EXTI Rising Edge	Direction toggle (internal pull-up enabled)
PA5	LD2_Pin	Output Push-Pull	Heartbeat / stop indicator LED

Note: PB15 is configured with GPIO_PULLUP in firmware (see MX_GPIO_Init in main.c, line 729). PC13 uses GPIO_NOPULL because the NUCLEO board provides an external pull-down on B1. Refer to the Troubleshooting section regarding external pull-up/pull-down resistor values for custom boards.

2.3 Peripheral Configuration

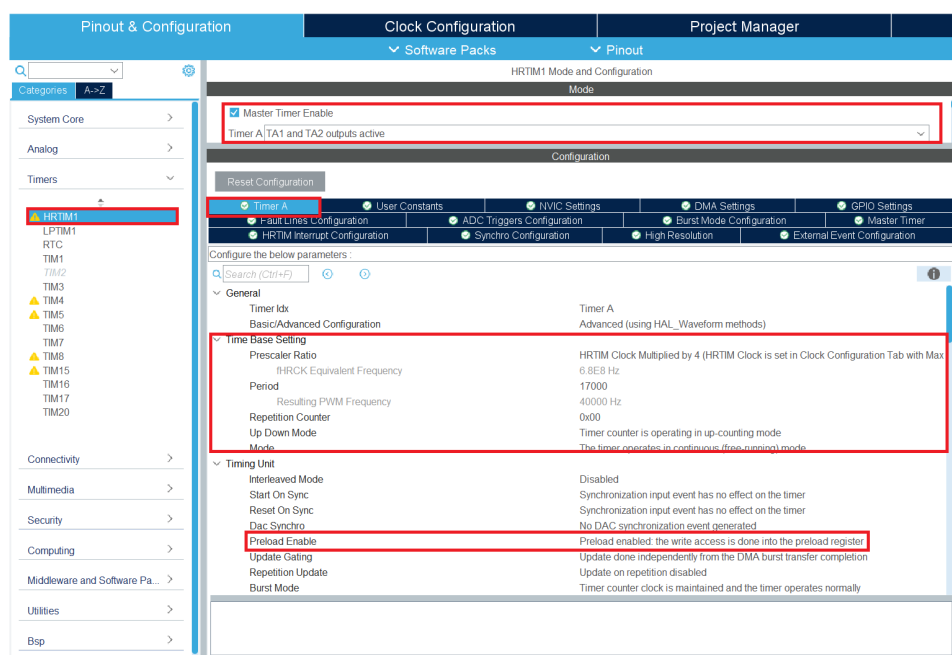


Figure 1: HRTIM master timer with $PER = 17000$ and preload register enabled.

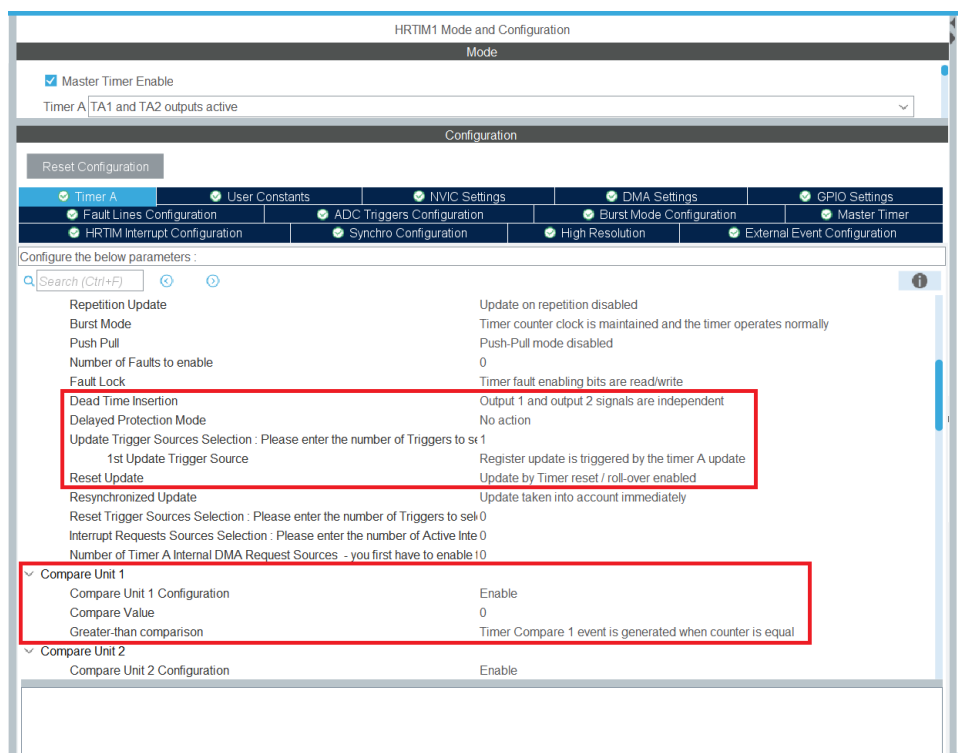


Figure 2: Timer A compare units CMP1 and CMP2 initialisation.

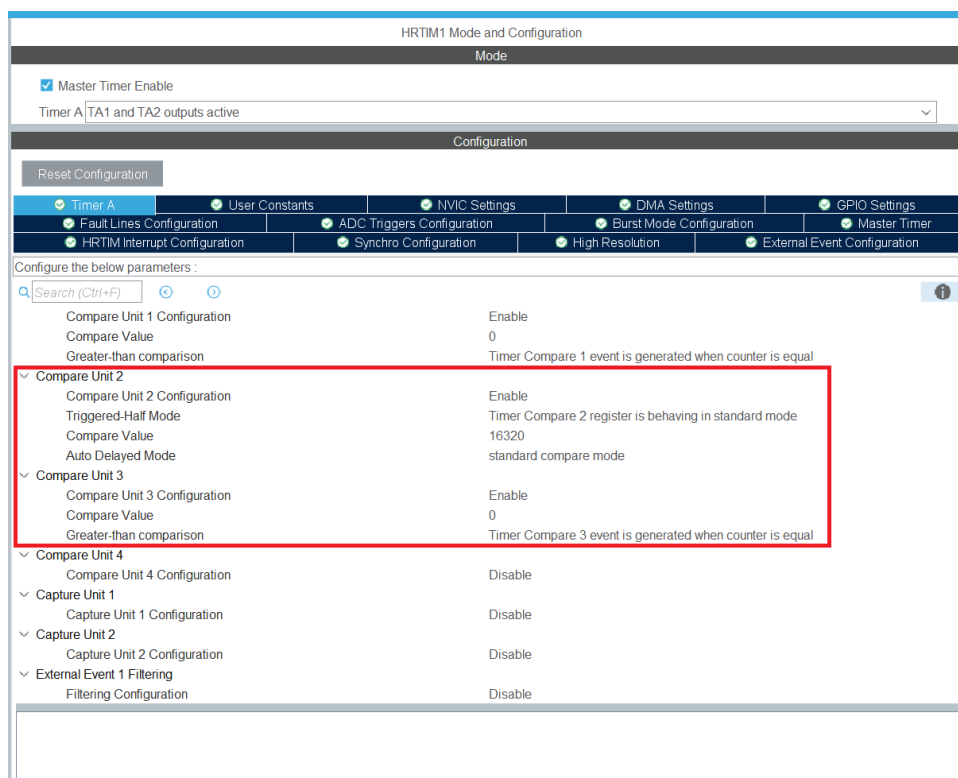


Figure 3: Timer A CMP3 and waveform mode continuation.

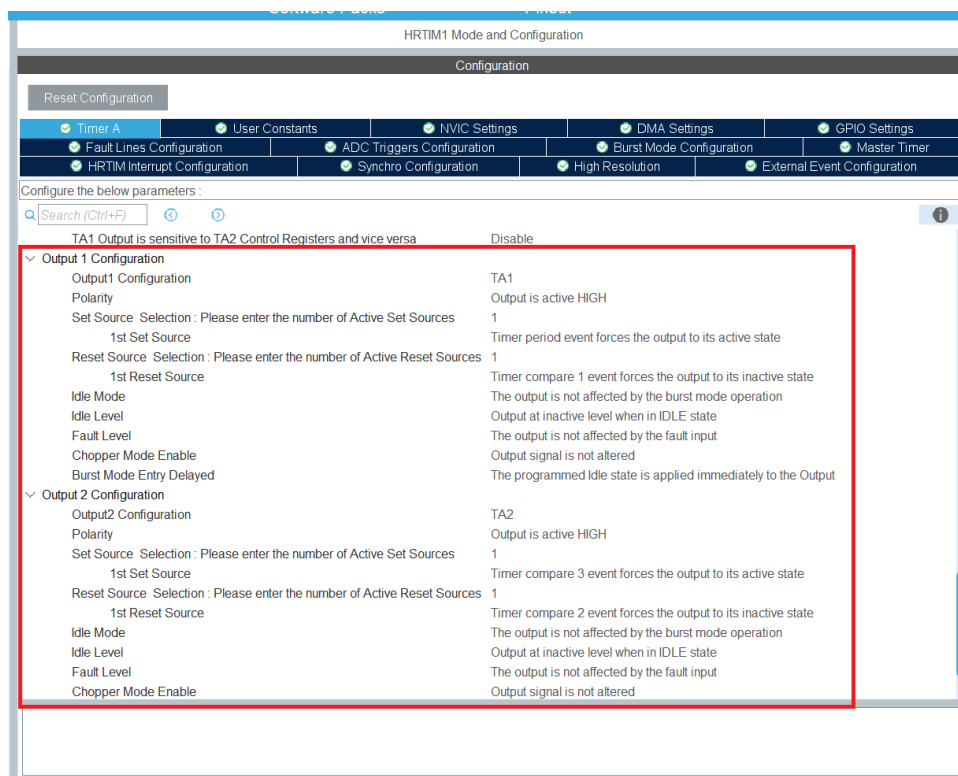


Figure 4: Output configuration: TA1 SET/RESET sources and TA2 SET/RESET sources.

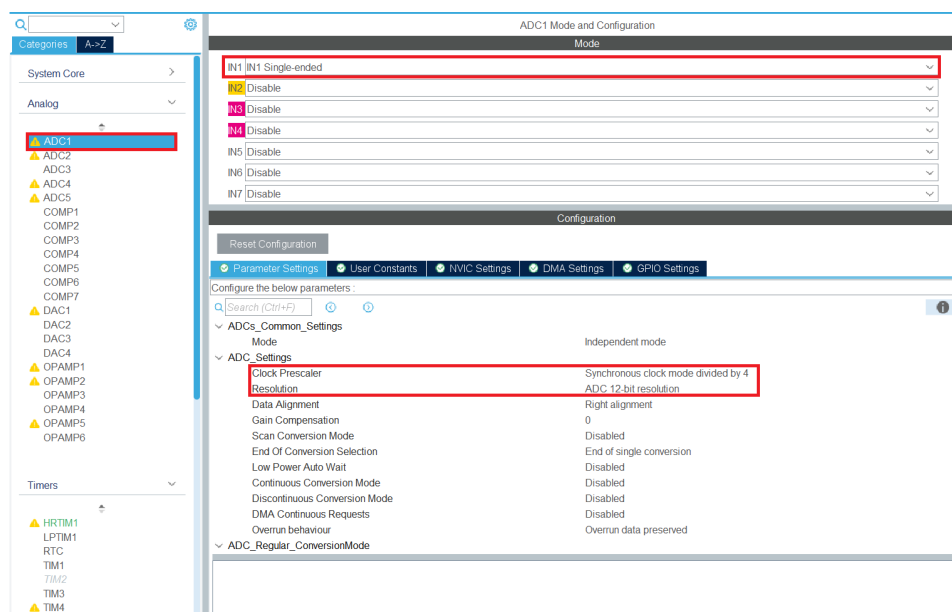


Figure 5: ADC1 clock prescaler set to PCLK/4.

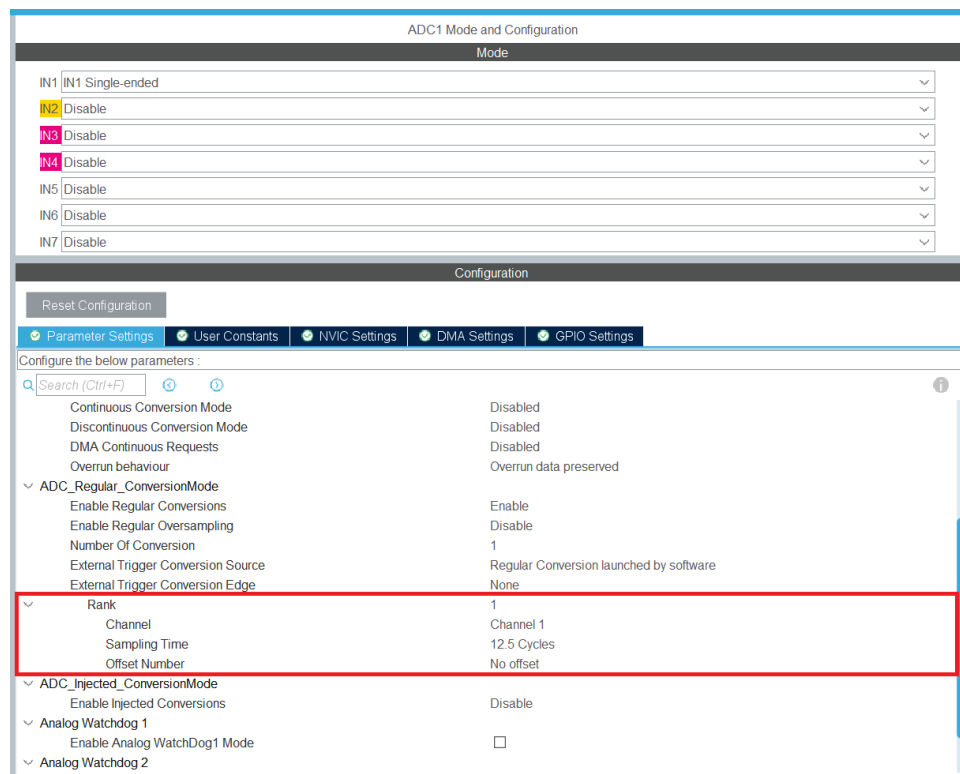


Figure 6: Channel 1 sample time: 12.5 ADC clock cycles.

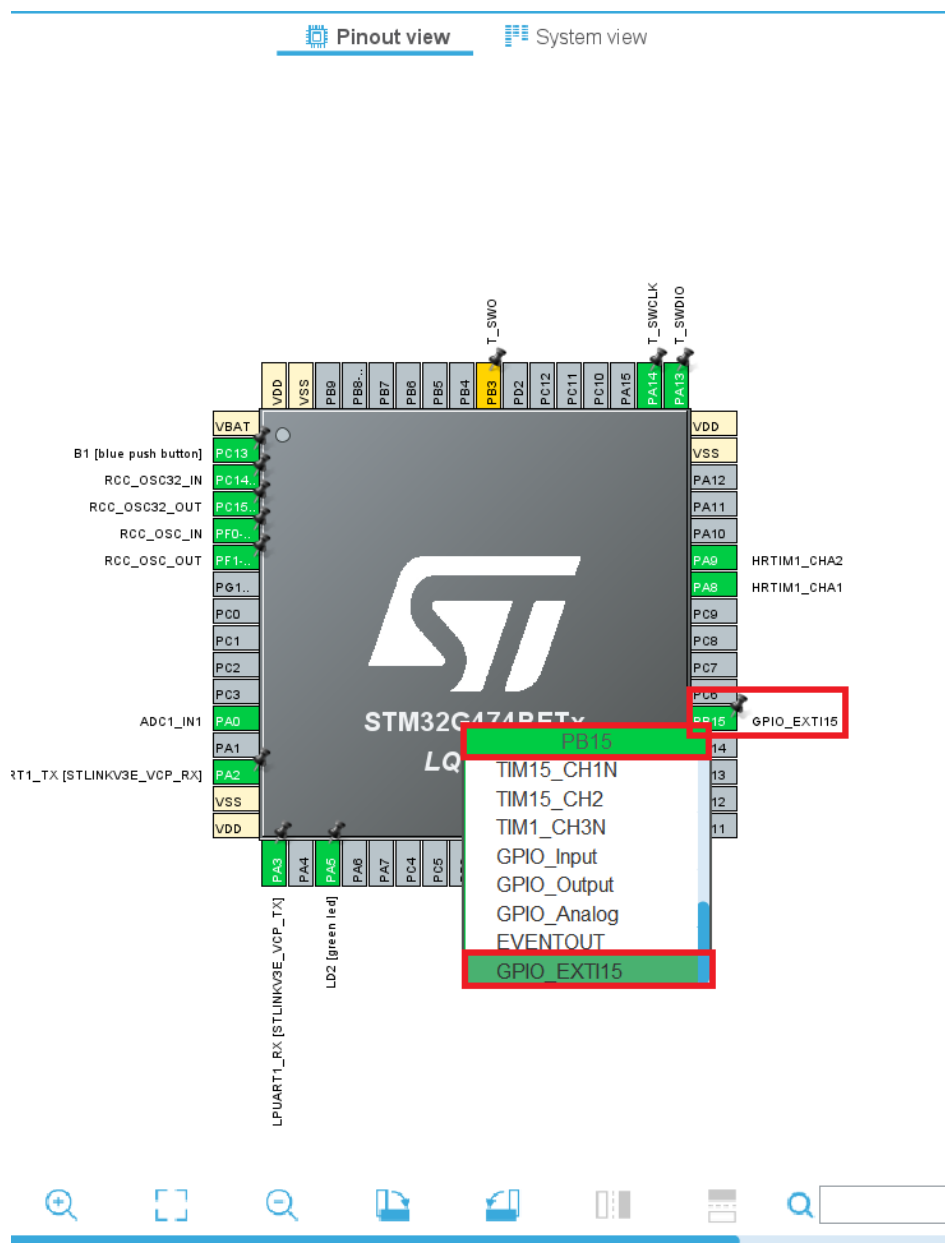


Figure 7: PB15 EXTI rising-edge with pull-up in CubeMX.

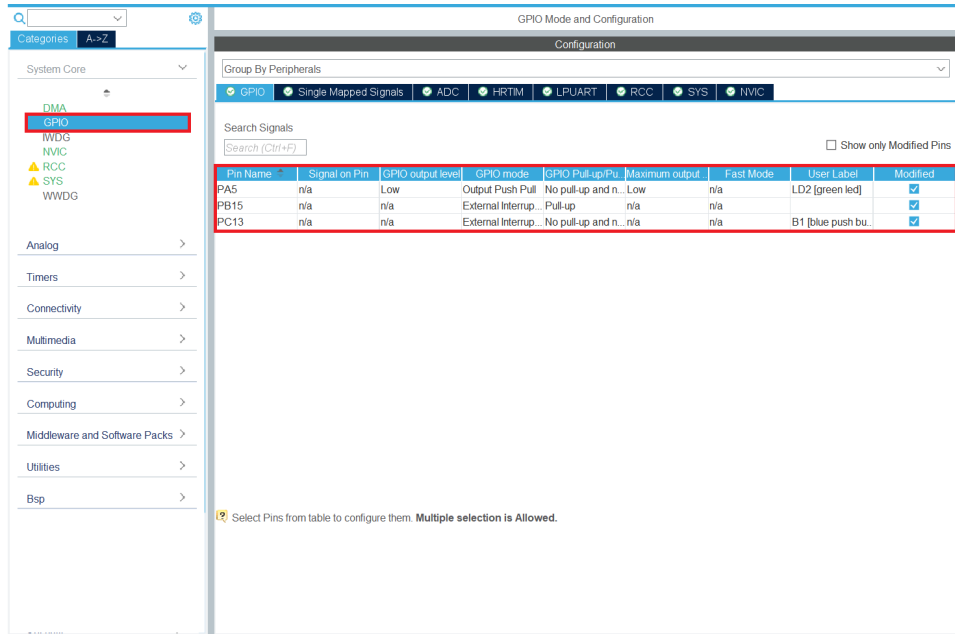


Figure 8: Full GPIO pin overview confirming all assignments.

2.4 Software Architecture

2.4.1 Project Structure

The STM32CubeIDE project generated for the NUCLEO-G474RE board follows a standard structure that separates device drivers, hardware abstraction layers, and application code:

```

1 CUBE_IDE/
2 |-- Core/
3 |   |-- Inc/
4 |   |   |-- main.h
5 |   |   '-- stm32g4xx_it.h
6 |   '-- Src/
7 |       |-- main.c
8 |       '-- stm32g4xx_it.c
9 |-- Drivers/
10 |   |-- STM32G4xx_HAL_Driver/
11 |   '-- CMSIS/
12 |-- Startup/
13 |   '-- startup_stm32g474xx.s
14 '-- .mxproject

```

Listing 1: STM32CubeIDE Project Directory Structure

The **Core/Src/main.c** file is where the primary application logic resides. STM32CubeIDE provides protected code regions marked with `/* USER CODE BEGIN */` and `/* USER CODE END */` comments, allowing developers to add custom code without it being overwritten during subsequent CubeMX code-generation cycles. The interrupt service routines reside entirely in **Core/Src/stm32g4xx_it.c**.

2.4.2 Initialisation Flow

On power-on, the following sequence executes before the control loop begins:

1. `HAL_Init()` — resets all peripherals, initialises the HAL tick timer (SysTick at 1 ms), and configures the NVIC priority grouping.
2. `SystemClock_Config()` — configures the HSE oscillator and PLL to produce a 170 MHz SYSCLK (PLLM=6, PLLN=85, PLLR=2, voltage scale BOOST).
3. `MX_GPIO_Init()` — enables port clocks, configures LD2 as push-pull output, B1 as EXTI rising-edge with no pull, and PB15 as EXTI rising-edge with internal pull-up. Both EXTI lines share EXTI15_10_IRQn at priority 0.
4. `MX_ADC1_Init()` — configures ADC1 in single-conversion software-triggered mode, 12-bit right-aligned, Channel 1, 12.5-cycle sample time.
5. `MX_HRTIM1_Init()` — runs DLL calibration, sets Timer A period to 17 000 ticks with MUL32 prescaler, configures CMP1/CMP2/CMP3, and assigns SET/RESET sources for TA1 and TA2.
6. `HAL_ADCEx_Calibration_Start()` — performs factory-recommended self-calibration for single-ended mode before the first conversion.
7. `Motor_Start()` — arms Timer A counter with near-zero compare values; duty climbs from zero on the first control tick.

2.5 Control Logic and FSM

The motor controller is a three-state FSM. State transitions are driven exclusively by the two EXTI callbacks and are communicated to the main loop via ISR-safe volatile boolean flags (`stopPend`, `dirTogglePend`) set in `HAL_GPIO_EXTI_Callback`.

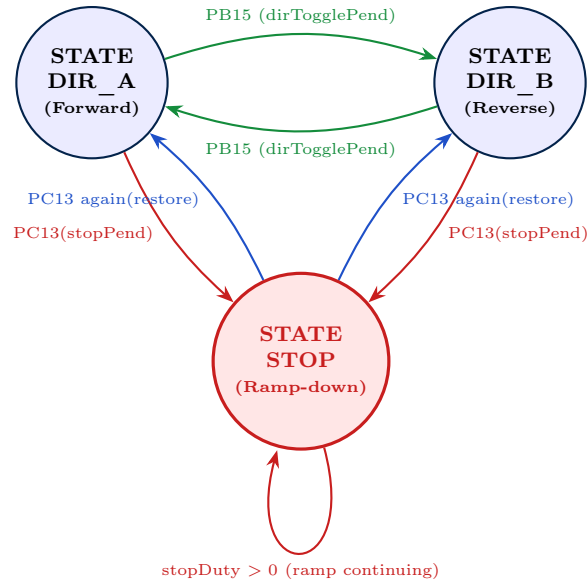


Figure 9: Motor Controller FSM. **Red**: stop/ramp-down. **Green**: direction toggle. **Blue**: resume after stop.

The direction toggle (PB15) is ignored while in `STATE_STOP`. The stop button (PC13) acts as a toggle: the first press initiates ramp-down; the second press restores the pre-stop direction and re-arms the HRTIM counter via `Motor_Start()`.

2.6 Code Implementation

2.6.1 Key Defines and Data Types

The following constants and type definitions (from `main.c`) form the numerical backbone of the controller:

```

1  /* MotorState_t      defined in USER CODE BEGIN PTD */
2  typedef enum {
3      STATE_DIR_A = 0,    /* Direction A: TA1 driven */
4      STATE_DIR_B = 1,    /* Direction B: TA2 driven (phase reversed) */
5      STATE_STOP  = 2     /* Controlled ramp-down, then disable */
6  } MotorState_t;
7
8  /* HRTIM timing constants      USER CODE BEGIN PD */
9  #define HRTIM_PERIOD_TICKS    17000U /* 680 MHz / 40 kHz */
10 #define HRTIM_DT_TICKS        680U /* 1 us * 680 MHz */
11 #define HRTIM_CMP2_TICKS      16320U /* PER - DT */
12 #define HRTIM_CMP_MIN         3U /* HW minimum */
13
14 /* Duty limits and ramp rate */
15 #define DUTY_MIN              0.0f
16 #define DUTY_MAX              1.0f
17 #define RAMP_RATE              0.001f /* delta-duty per 10 ms tick */
18 #define ADC_FULL_SCALE        4095.0f

```

```

19
20 /* Clamp macro */
21 #define CLAMP_F(x, lo, hi) \
22     ( (x)<(lo)?(lo):( (x)>(hi)?(hi):(x) ) )

```

Listing 2: HRTIM Timing Constants and State Enumeration (`main.c`)

2.6.2 Main Initialisation

The `main()` entry point (from `main.c`) follows the standard CubeMX-generated sequence with user additions in the protected regions:

```

1 int main(void)
2 {
3     HAL_Init();
4     SystemClock_Config();
5     MX_GPIO_Init();
6     MX_ADC1_Init();
7     MX_HRTIM1_Init();
8
9     /* USER CODE BEGIN 2 */
10    /* ADC self-calibration (single-ended) */
11    HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
12
13    /* Arm HRTIM counter with near-zero duty */
14    Motor_Start();
15    /* USER CODE END 2 */
16
17    while (1)
18    {
19        /* Control loop      see Core Functionality */
20    }
21 }

```

Listing 3: Application Entry Point and Startup Sequence (`main.c`)

2.6.3 Core Functionality

The infinite loop runs at a 10 ms cadence enforced by `HAL_Delay(10)`. Each tick performs four steps:

1. **Direction toggle service:** If `dirTogglePend` is set (and not in `STATE_STOP`), the state flips between `STATE_DIR_A` and `STATE_DIR_B` and the flag is cleared.
2. **Stop/resume service:** If `stopPend` is set, it toggles between entering `STATE_STOP` (snapshotting `masterDuty` into `stopDuty`) and resuming the pre-stop direction.
3. **ADC read:** A software-triggered single conversion on ADC1 CH1 produces `adcDuty` $\in [0, 1]$.

4. **Output drive:** Depending on state, either `Motor_SetDirA`, `Motor_SetDirB`, or the ramp-down path drives the HRTIM outputs.

```

1 while (1)
2 {
3     /* 1. Direction toggle */
4     if (dirTogglePend && (motorState != STATE_STOP))
5     {
6         dirTogglePend = false;
7         motorState = (motorState == STATE_DIR_A)
8                     ? STATE_DIR_B : STATE_DIR_A;
9         dirBeforeStop = motorState;
10    }
11
12    /* 2. Stop / resume */
13    if (stopPend)
14    {
15        stopPend = false;
16        if (motorState == STATE_STOP)
17        {
18            motorState = dirBeforeStop;
19            HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
20                             GPIO_PIN_RESET);
21        }
22        else
23        {
24            dirBeforeStop = motorState;
25            stopDuty       = masterDuty;
26            motorState     = STATE_STOP;
27        }
28    }
29
30    /* 3. Read ADC */
31    float adcDuty = ADC_ReadDutyReference();
32
33    /* 4. Apply duty */
34    if (motorState == STATE_STOP)
35    {
36        stopDuty -= RAMP_RATE;
37        if (stopDuty < 0.0f) stopDuty = 0.0f;
38        if (stopDuty <= 0.0f)
39        {
40            Motor_DisableAll();
41            HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
42                             GPIO_PIN_SET);
43        }
44        else
45        {

```

```

46         if (dirBeforeStop == STATE_DIR_A)
47             Motor_SetDirA(stopDuty);
48         else
49             Motor_SetDirB(stopDuty);
50     }
51 }
52 else if (motorState == STATE_DIR_A)
53 {
54     masterDuty = adcDuty;
55     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
56                       GPIO_PIN_RESET);
57     Motor_SetDirA(masterDuty);
58 }
59 else /* STATE_DIR_B */
60 {
61     masterDuty = adcDuty;
62     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,
63                       GPIO_PIN_RESET);
64     Motor_SetDirB(masterDuty);
65 }
66
67 HAL_Delay(10);
68 }

```

Listing 4: Main Control Loop with State Machine (`main.c`)

2.6.4 HRTIM Output Helpers

Two direction-specific functions configure the HRTIM compare registers and gate the appropriate output:

Table 2: HRTIM Output Configuration per Direction

Function	Motor_SetDirA (TA1 active)	Motor_SetDirB (TA2 active)
Active output	TA1 started; TA2 stopped	TA2 started; TA1 stopped
CMP1 role	TA1 RESET point = $\text{duty} \times \text{PER}$	Used to compute CMP3
CMP3 role	Set to $\text{CMP_MIN} + \text{DT_TICKS}$ (start value)	TA2 SET = $\text{CMP1} + 680$ ticks
CMP2 role	Fixed 16 320 (end-of-cycle DT)	Fixed 16 320 (TA2 RESET)
Duty complement	TA2 is idle LOW	Effective duty = $(16320 - \text{CMP3})/17000$

2.6.5 Interrupt Service Routines

The ISR strategy is deliberately minimal: only volatile boolean flags are written inside callbacks, eliminating any risk of HAL re-entrancy or priority inversion.

```

1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {

```



```

3     if (GPIO_Pin == B1_Pin)
4         stopPend = true;           /* PC13 -- stop or resume */
5
6     if (GPIO_Pin == GPIO_PIN_15)
7         dirTogglePend = true;     /* PB15 -- direction latch toggle */
8 }

```

Listing 5: EXTI Callback — Flag-Only ISR Pattern (`main.c`)

The shared EXTI15_10 vector is dispatched in `stm32g4xx_it.c`. Both pin lines are cleared by the HAL dispatcher before the callback fires:

```

1 void EXTI15_10_IRQHandler(void)
2 {
3     HAL_GPIO_EXTI_IRQHandler(B1_Pin);
4     HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_15);
5 }

```

Listing 6: Shared EXTI Vector Dispatch (`stm32g4xx_it.c`)

The remaining handlers in `stm32g4xx_it.c` (NMI, HardFault, MemManage, BusFault, UsageFault) all loop indefinitely, and `SysTick_Handler` calls `HAL_IncTick()` to maintain the HAL millisecond timebase used by `HAL_Delay()`.

Note: The ISR execution time for both `stopPend` and `dirTogglePend` flag writes is under 10 clock cycles. No HAL peripheral calls, floating-point operations, or state writes are performed inside the ISR context. This is the recommended pattern for HRTIM-class applications where timing jitter from ISR overhead could affect switching regularity.

2.7 Debugging

Exploration and Learning: The most effective way to explore this project is to vary the potentiometer and observe the PWM duty cycle on an oscilloscope or logic analyser connected to PA8 (TA1) and PA9 (TA2). Press PC13 and watch the duty ramp to zero; press it again to resume. Toggle PB15 and confirm that TA1 and TA2 swap active roles.

Using the Integrated Debugger: If you wish to use the STM32CubeIDE live-expression debugger:

1. Set a breakpoint at `HAL_Delay(10)` inside the while loop.
2. Add `masterDuty`, `stopDuty`, and `motorState` to the Live Expressions window.
3. Resume execution and interact with the buttons; the expressions update on each breakpoint hit.
4. To inspect HRTIM register values directly, open *Window* → *Show View* → *Registers* and expand the HRTIM1 peripheral tree.

Note: If the motor or H-bridge does not respond, verify that `Motor_Start()` was called and that the HRTIM output state returned by `HAL_HRTIM_WaveformGetOutputState` is `HRTIM_OUTPUTSTATE_RUN` for the active channel. A common issue is forgetting to call `Motor_Start()` after `Motor_DisableAll()` when resuming from stop.

3 MATLAB Embedded Coder Implementation

3.1 Prerequisites

Before starting the MATLAB Embedded Coder implementation, ensure you have gone through all the previous projects in the series at NUCLEO-G474RE-PowerElectronics-Guide, particularly the HRTIM and ADC standalone modules.

Note: The ARM GCC compiler configuration is critical for successful code generation. By default, MATLAB may be configured with a different toolchain. This must be changed to **GNU Tools for ARM Embedded Processors** for STM32 compatibility.

3.2 MATLAB/Simulink Model Design

3.2.1 Model Architecture

The Simulink model replicates the three-state control FSM of the CubeIDE implementation using Embedded Coder-compatible library blocks. The top-level signal flow is:

1. An **ADC block** (Embedded Coder STM32 support package) reads the potentiometer on ADC1 CH1 and outputs a raw 12-bit integer, which is normalised to $[0, 1]$ by a **Gain** block ($1/4095$).
2. Two **ISR Triggered Subsystems** capture the rising-edge events on PC13 (stop) and PB15 (direction), setting state latches implemented as **Unit Delay** blocks.
3. A pair of **Multipoint Switches** route the duty-cycle value to the correct HRTIM output channel based on the direction latch state. When `Dir = 0`, the switch passes the duty to the TA1 HRTIM block; when `Dir = 1`, it routes to the TA2 block.
4. A **Discrete-Time Integrator** implements the master ramp integrator. Its input is switched between the normalised ADC value (running) and the constant 0 (stopping) by the stop-state latch. The integrator output represents `masterDuty`.
5. Two **Saturation** blocks clamp the integrator output and the direct ADC path to $[0, 1]$ before writing to the HRTIM compare registers.
6. **HRTIM blocks** (TA1 and TA2) accept the clamped duty value and update CMP1 and CMP3 respectively each sample step.

3.2.2 Block Functionality

Table 3: Simulink Block Summary and Rationale

Block	Library	Role in this Model
ADC	STM32 Embedded Coder Support	Reads potentiometer; outputs 12-bit raw integer
Gain ($\times 1/4095$)	Simulink/Math Operations	Normalises ADC count to floating-point $[0, 1]$ duty
ISR Triggered Subsystem	Embedded Coder STM32	Captures GPIO EXTI events for PC13 and PB15 asynchronously
Unit Delay	Simulink/Discrete	Latches direction and stop flags between sample steps
Multiport Switch	Simulink/Signal Routing	Routes duty signal to TA1 or TA2 based on direction latch
Discrete-Time Integrator	Simulink/Discrete	Ramp integrator; input is 0 when stop active, ADC duty otherwise
Saturation	Simulink/Discontinuities	Clamps duty to $[0, 1]$; essential safety clamp for power outputs
HRTIM (TA1 / TA2)	STM32 Embedded Coder Support	Writes compare register to set PWM duty on each output
Constant (0)	Simulink/Sources	Stop input to integrator; drives duty toward zero during ramp-down

The **Multiport Switch** is the key architectural block: it does not change the duty value itself but selects *which* physical HRTIM output receives it, achieving bidirectional drive without re-computing the ramp. The **Saturation** blocks enforce hard limits on all paths into the HRTIM, mirroring the `CLAMP_F` macro in the CubeIDE firmware. In power electronic and industrial control systems, these saturation/clamp stages are non-negotiable safety elements—they prevent compare registers from overflowing the period and causing undefined switching behaviour.

3.3 Model Configuration

The Simulink model is located at `05_H_Bridge_Control/MATLAB_Coder/`. The block diagram overview is shown in `Block_diagram.png` (top-level model view for reference).

3.3.1 CubeMX Configuration

The CubeMX peripheral settings used for the MATLAB Embedded Coder flow are identical to those of the CubeIDE implementation. The same screenshot set applies:

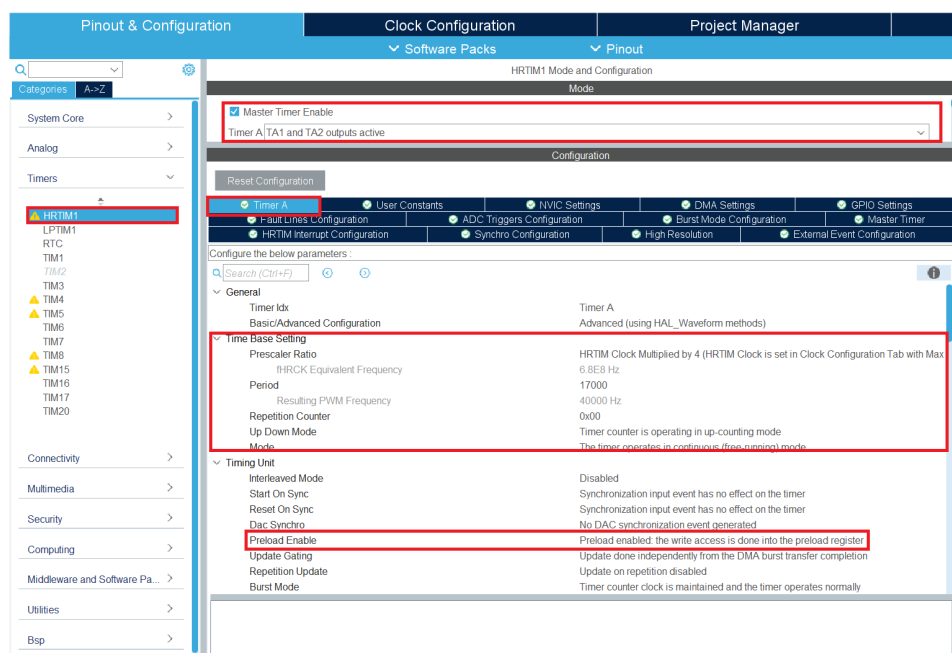


Figure 10: Period = 17 000, preload enabled (shadow-to-active handshake on counter reset).

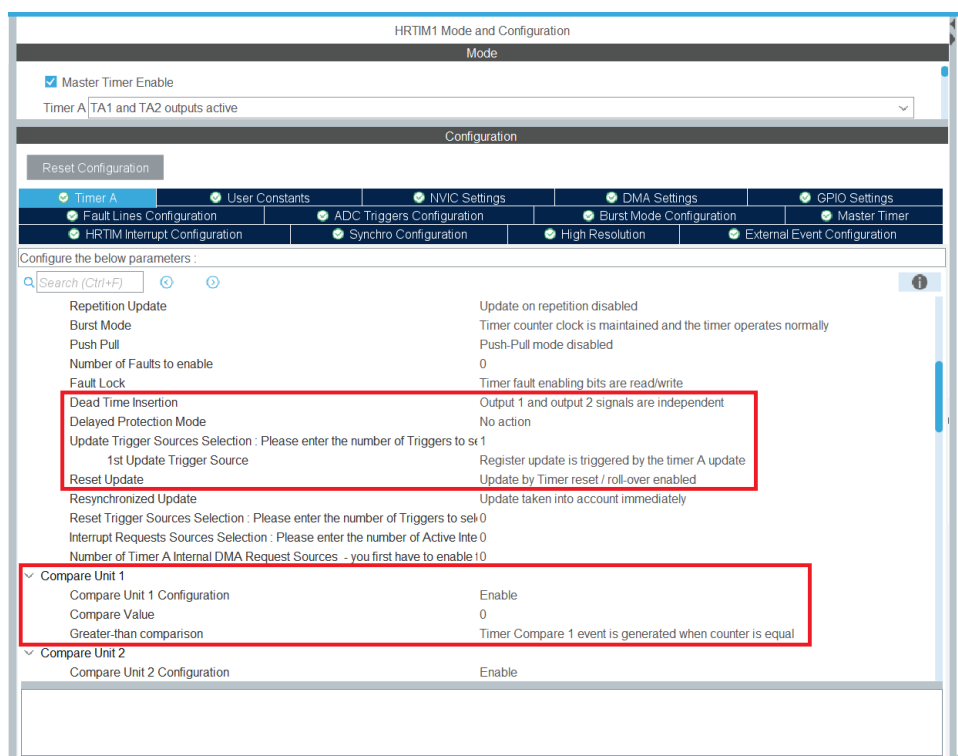


Figure 11: CMP1 initial value = 0; CMP2 = 16 320.

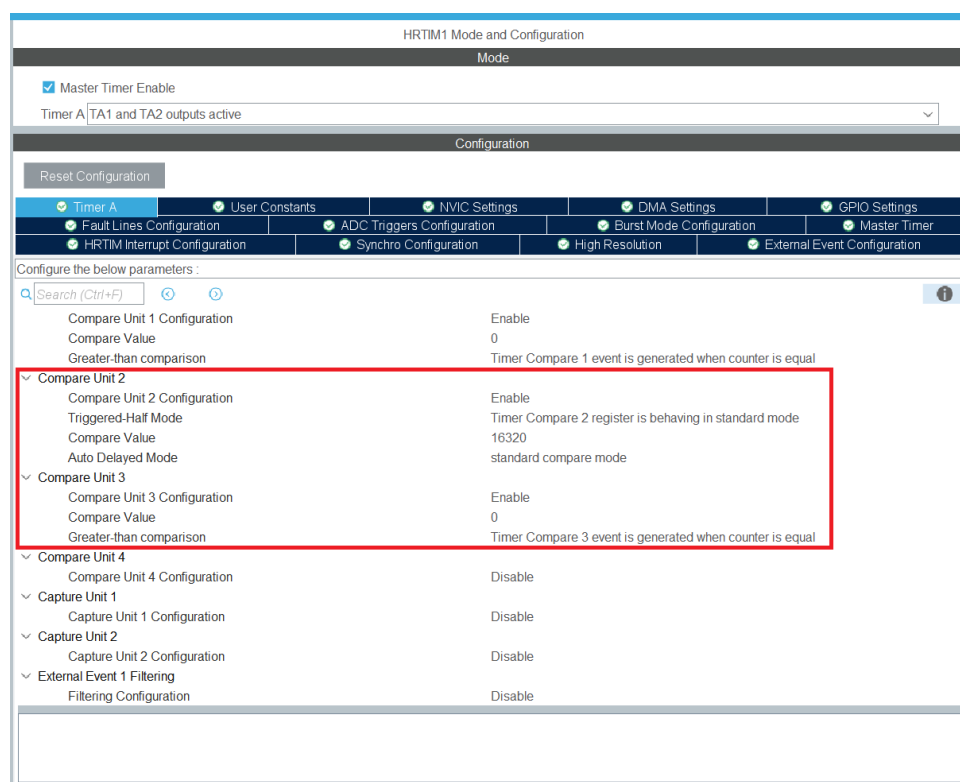


Figure 12: CMP3 initial value = 0; Timer A in continuous up-counting mode with MUL32 prescaler.

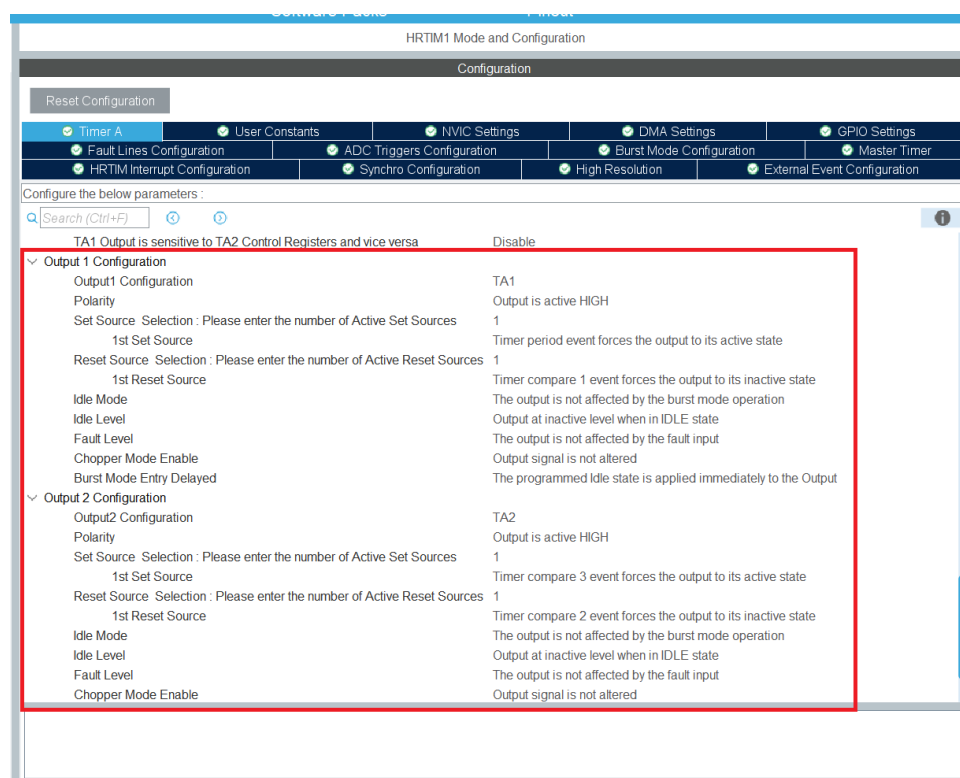


Figure 13: TA1: SET = Period event, RESET = CMP1. TA2: SET = CMP3, RESET = CMP2.

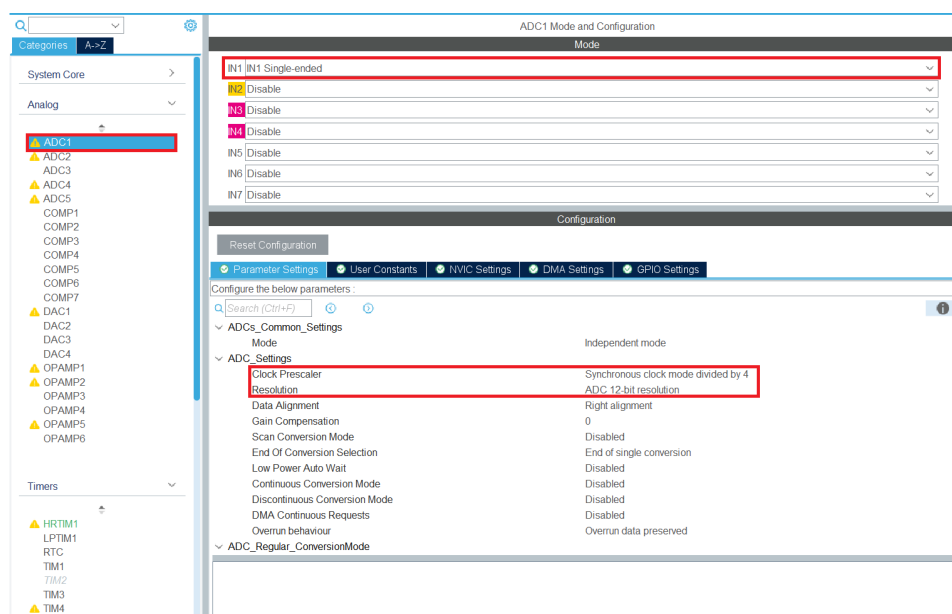


Figure 14: ADC1, PCLK/4 synchronous clock.

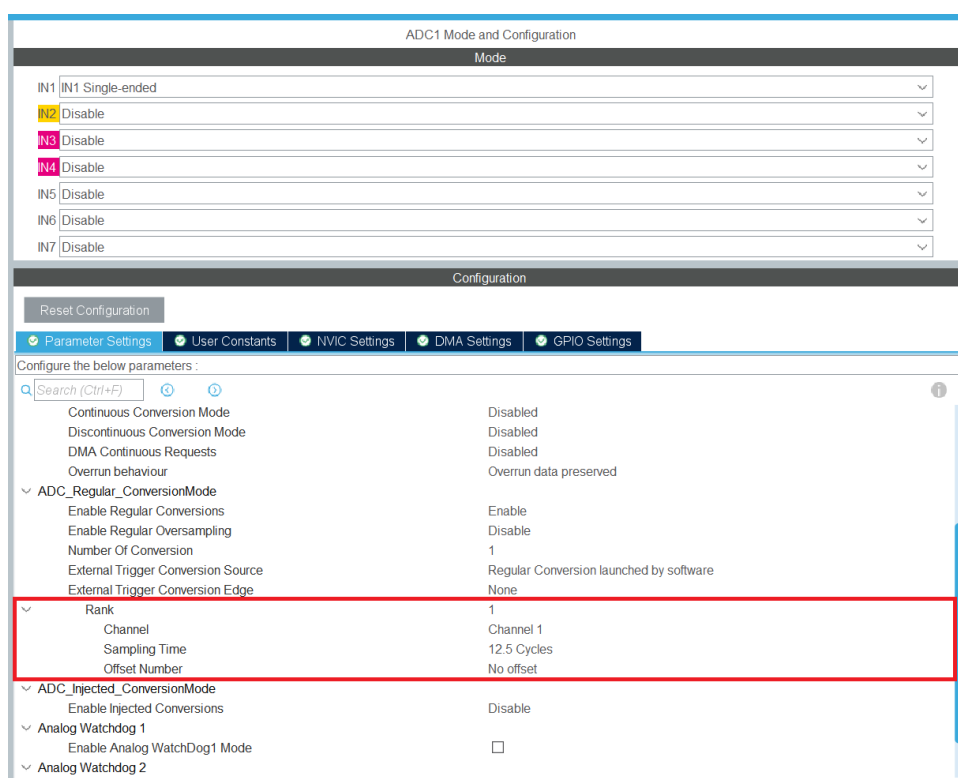


Figure 15: Channel 1, 12.5-cycle sample time, single-ended, software trigger.



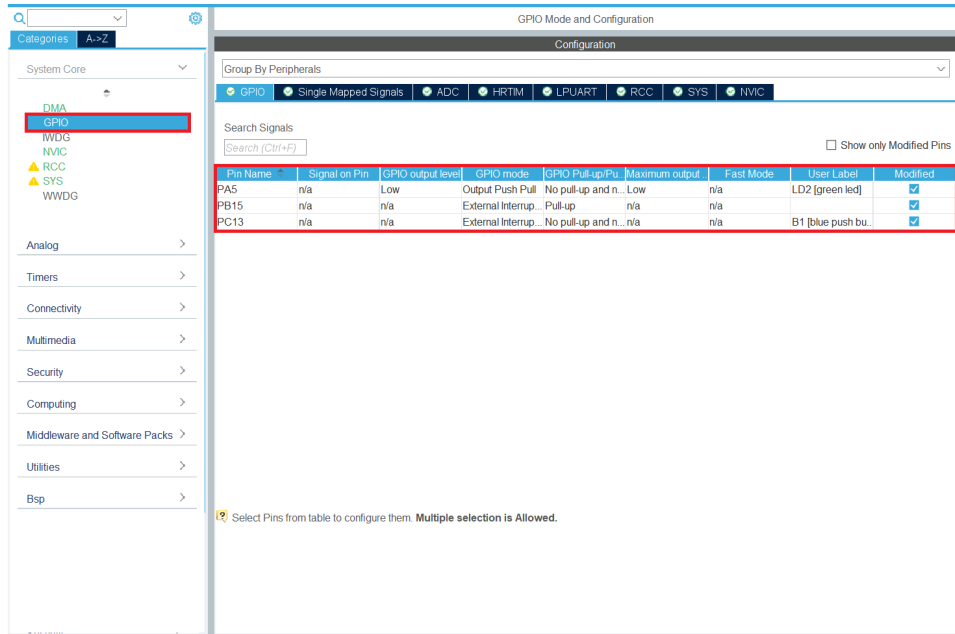


Figure 17: Full pin assignment verification.

4 STM32CubeIDE vs. MATLAB Embedded Coder

4.1 Implementation Comparison

Table 4: Side-by-Side Implementation Comparison

Aspect	STM32CubeIDE	MATLAB Embedded Coder
Control loop	Hand-written C, 10 ms HAL_Delay tick	Simulink sample time, auto-scheduled
State machine	Explicit enum + ISR flags	Multiport Switch + Unit Delay latches
Ramp integrator	Manual float accumulator with clamp macro	Discrete-Time Integrator + Saturation block
Safety clamps	CLAMP_F macro on all duty paths	Saturation blocks on every HRTIM input
ISR handling	HAL_GPIO_EXTI_Callback flag-only	ISR Triggered Subsystem in Simulink
HRTIM update	Direct HAL register writes	HRTIM Embedded Coder block
Portability	Full register-level control	Rapid model-based iteration

4.2 Key Differences in this Project

STM32CubeIDE Implementation: Offers explicit cycle-accurate control over every HRTIM register and ISR. The guard conditions (e.g., `cmp3Ticks` clamped below `HRTIM_PERIOD_TICKS`) are hand-coded and fully transparent. Ideal for production code where every clock matters.

MATLAB Embedded Coder Implementation: The model-based approach makes the signal flow visually inspectable. The Discrete-Time Integrator and Saturation blocks directly mirror the mathematical specification, making the model a living piece of documentation. Rapid

iteration and parameter sweeping are significantly easier.

4.3 Hybrid Approach Recommendation

Use MATLAB/Simulink for algorithm prototyping and verification of the ramp integrator dynamics and saturation limits. Once timing is validated, the CubeIDE implementation provides the production-grade register-level precision required for high-frequency switching applications.

5 Testing and Validation

5.1 Visual Verification

Connect an oscilloscope to PA8 (TA1) and PA9 (TA2). At mid-scale potentiometer, expect: a 40 kHz waveform with approximately 50% duty on the active channel, the complementary channel inverted, and a visible 1 μ s dead-band on both rising and falling edges where neither output is active.

5.2 Functional Testing

1. **ADC tracking:** Rotate the potentiometer full-scale. The active output duty should sweep smoothly from near-zero to near-100%.
2. **Direction toggle (PB15):** Press PB15. TA1 should go idle and TA2 should become the driven output (or vice versa). The duty level should be unchanged.
3. **Stop ramp (PC13):** Press PC13. The active duty should visibly ramp down to zero over approximately $1/(0.001 \times 100) = 10$ s (at full duty). LD2 illuminates when ramp completes.
4. **Resume (PC13 again):** Press PC13 a second time. The pre-stop direction should restore and duty should track the ADC immediately.

5.3 Performance Characteristics

Table 5: Key Performance Parameters

Parameter	Value	Source
Switching frequency	40 kHz	PER = 17 000 ticks @ 680 MHz
Dead-time	1 μ s	680 ticks @ 680 MHz
ADC resolution	12-bit (4096 steps)	ADC_RESOLUTION_12B
Control loop period	10 ms	HAL_Delay(10)
Ramp step per tick	0.001 (normalised duty)	RAMP_RATE define

6 Results and Analysis

6.1 Expected Outcomes

STM32CubeIDE Implementation Result:

The STM32CubeIDE build produces complementary PWM waveforms on TA1/TA2 with correct dead-time on both edges. Direction toggling is instantaneous (next control tick) with no glitch on the complementary channel. The ramp-down correctly decelerates the duty from the running level to zero, and LD2 provides a clear visual confirmation of the stopped state. The demonstration video is available at:

https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/blob/main/Examples/05_H_Bridge_Control//H_Bridge_STM.mp4

MATLAB Embedded Coder Implementation Result:

The Embedded Coder model generates equivalent behaviour. The Discrete-Time Integrator visibly ramps the duty on the Simulink scope before deployment, and the same oscilloscope waveforms are obtained post-flash. The demonstration video is available at:

https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/blob/main/Examples/05_H_Bridge_Control//H_Bridge_Matlab.mp4

6.2 Key Observations

- The 1 μ s dead-time is clearly measurable on a oscilloscope as a brief period where both TA1 and TA2 are simultaneously LOW on both switching edges.
- Direction reversal does not require a duty recalculation—the Multiport Switch / direction-gating pattern is both simpler and safer than independently computing two duty values.
- At `RAMP_RATE = 0.001` and 10 ms ticks, a ramp from full duty (≈ 1.0) to zero takes approximately 10 s, which is conservative but illustratively clear. In a real motor drive this would be tuned to the mechanical time constant.
- LD2 provides an unambiguous status indicator: OFF during normal operation (any direction), ON when the motor is stopped.

6.3 Project Validation

The project is considered validated when all four functional tests pass: ADC duty sweep, direction toggle with correct channel swap, ramp-down to zero with LD2 illumination, and clean resume after a second PC13 press. Both CubeIDE and MATLAB implementations must produce identical oscilloscope signatures to confirm that the Embedded Coder code generation correctly maps the Simulink blocks to the underlying HRTIM HAL calls.

7 Troubleshooting

7.1 Common Issues and Solutions

Table 6: Common Issues and Recommended Fixes

Symptom	Likely Cause	Fix
Button not responding	Floating GPIO input; no pull resistor	Use internal pull-up (<code>GPIO_PULLUP</code>) or add external pull-up (4.7–10 k Ω to VDD) for active-low, or pull-down (4.7–10 k Ω to GND) for active-high configuration
Multiple triggers per press	Mechanical contact bounce	Add software debounce delay or use a 100 nF capacitor across the button terminals
No PWM output	<code>Motor_Start()</code> not called	Ensure <code>Motor_Start()</code> is called after <code>MX_HRTIM1_Init()</code> and after any <code>Motor_DisableAll()</code>
Shoot-through (both switches ON)	Dead-time too small or misconfigured	Verify <code>CMP3 = CMP1 + 680</code> and <code>CMP2 = 16320</code> ; check oscilloscope for overlap
Duty stuck at zero	ADC not calibrated	Call <code>HAL_ADCEX_Calibration_Start</code> before first conversion
HardFault on resume	HRTIM counter not re-armed	Call <code>Motor_Start()</code> inside the resume branch of the stop toggle

7.2 Debugging Recommendations

Use the STM32CubeIDE Register view to inspect live HRTIM1 compare register values (`CMP1xR`, `CMP2xR`, `CMP3xR`) and confirm they match calculated tick counts. Enable the SWO trace output for `printf` logging of `masterDuty` and `motorState` without halting execution.

8 Extensions and Enhancements

8.1 Further Learning

- **Current sensing:** Add a shunt resistor in the low-side leg and route the amplified signal to a second ADC channel. Implement a simple PI current controller in the control loop.
- **Closed-loop speed control:** Connect an encoder to a timer in quadrature mode and implement a PID speed loop that drives the duty-cycle integrator setpoint.
- **Fault injection:** Explore the HRTIM fault input (`HRTIM_FLT`) to implement hardware-level shoot-through protection that cuts both outputs instantly on an overcurrent signal.

- **Tunable ramp rate:** Expose `RAMP_RATE` as a parameter read from a second potentiometer via ADC2, enabling run-time adjustable deceleration slope.

9 Conclusion

9.1 Key Takeaways

- The HRTIM's DLL-multiplied 680 MHz clock enables dead-time resolution of 1.47 ns per tick—far finer than any software timer could achieve.
- Separating ISR context (flag writes only) from main-loop context (all state changes and HAL calls) is the correct architectural pattern for high-frequency power electronic firmware.
- A ramp integrator with output saturation is the minimum viable safety mechanism for any motor drive; abrupt duty step changes risk destructive back-EMF and inrush current.
- The CubeIDE and MATLAB Embedded Coder implementations are architecturally isomorphic—understanding one maps directly to understanding the other.

9.2 Next Steps

- Integrate a current-sense amplifier and implement inner-loop current control for torque regulation.
- Add an RTOS (FreeRTOS via STM32CubeIDE middleware) to separate the communication, sensing, and control tasks into independent threads, providing a more cohesive control package which would further include display and communication.
- Migrate the Simulink model to use *Stateflow* for the FSM to gain formal state-chart verification tooling.

References

References

- [1] STMicroelectronics, *STM32G4 Series Advanced ARM-based 32-bit MCUs Reference Manual*, RM0440.
- [2] STMicroelectronics, *STM32G474RE Datasheet*.
- [3] STMicroelectronics, *STM32G4 NUCLEO-64 Boards User Manual*, UM2505.
- [4] STMicroelectronics, *STM32CubeIDE Integrated Development Environment User Manual*, DM00629855.
- [5] STMicroelectronics, *Description of STM32G4 HAL and Low-Layer Drivers*, UM2570.
- [6] STMicroelectronics, *HRTIM Cookbook*, AN4539.
- [7] MathWorks, *Embedded Coder Documentation*.
- [8] MathWorks, *Embedded Coder Support Package for STMicroelectronics STM32*.

A HRTIM PWM Configuration Register Summary

This appendix documents the key HRTIM1 Timer A register values as configured by `MX_HRTIM1_Init()` in `main.c`.

Table 7: HRTIM Timer A Time-Base Configuration

Parameter	Value	Remarks
PrescalerRatio	HRTIM_PRESCALERRATIO_MUL32	DLL $\times 4$ effective (HAL maps MUL32 to $\times 4$ for G4) giving 680 MHz
Period	17 000	$f_{HRCK}/f_{PWM} = 680 \text{ MHz}/40 \text{ kHz}$
Mode	HRTIM_MODE_CONTINUOUS	Free-running; counter wraps at period
PreloadEnable	HRTIM_PRELOAD_ENABLED	Shadow registers; active update on reset
ResetUpdate	HRTIM_TIMUPDATEONRESET_ENABLED	Triggers shadow-to-active transfer on counter wrap
UpdateTrigger	HRTIM_TIMUPDATETRIGGER_TIMER_A	Self-update on Timer A period

Table 8: HRTIM Timer A Compare Register Assignments

Register	Init Value	Runtime Value	Function
CMP1	0	duty $\times 17\,000$	TA1 RESET (high-side turn-off)
CMP2	16 320	Fixed 16 320	TA2 RESET (end-of-cycle DT; = PER – DT)
CMP3	0	CMP1 + 680	TA2 SET (low-side turn-on after 1 μ s DT)

Table 9: HRTIM Timer A Output Event Sources

Output	SET Source	RESET Source	Remarks
TA1	HRTIM_OUTPUTSET_TIMPER	HRTIM_OUTPUTRESET_TIMCMP1	High-side: ON at $t = 0$, OFF at CMP1
TA2	HRTIM_OUTPUTSET_TIMCMP3	HRTIM_OUTPUTRESET_TIMCMP2	Low-side: ON at CMP3, OFF at CMP2

B ADC1 Configuration Summary

Table 10: ADC1 Initialisation Parameters

Parameter	Value	Remarks
Instance	ADC1	Primary ADC
ClockPrescaler	ADC_CLOCK_SYNC_PCLK_DIV4	Synchronous PCLK/4; PCLK=170 MHz $\Rightarrow$ ADC CLK=42.5 MHz
Resolution	ADC_RESOLUTION_12B	4096 counts; 0–3.3 V $\Rightarrow$ 0–4095
DataAlign	ADC_DATAALIGN_RIGHT	LSB-aligned result
ScanConvMode	ADC_SCAN_DISABLE	Single channel
ContinuousConvMode	DISABLE	Software-triggered single conversion
ExternalTrigConv	ADC_SOFTWARE_START	Triggered by HAL_ADC_Start()
Channel	ADC_CHANNEL_1	PA1 (potentiometer input)
SamplingTime	ADC_SAMPLETIME_12CYCLES_5	12.5 ADC clock cycles
SingleDiff	ADC_SINGLE_ENDED	Single-ended measurement

C Full Source Code Listings

```

1 static void Motor_SetDirA(float duty)
2 {
3     duty = CLAMP_F(duty, DUTY_MIN, DUTY_MAX);
4     HRTIM_CompareCfgTypeDef cmp = {0};
5     cmp.CompareValue = Duty_ToTicks(duty);
6     HAL_HRTIM_WaveformCompareConfig(&hhrtim1,
7         HRTIM_TIMERINDEX_TIMER_A, HRTIM_COMPAREUNIT_1, &cmp);
8     HAL_HRTIM_WaveformOutputStop(&hhrtim1, HRTIM_OUTPUT_TA2);
9     HAL_HRTIM_WaveformOutputStart(&hhrtim1, HRTIM_OUTPUT_TA1);
10 }

```

Listing 7: Motor_SetDirA — Direction A Output Drive (main.c)

```

1 static void Motor_SetDirB(float duty)
2 {
3     duty = CLAMP_F(duty, DUTY_MIN, DUTY_MAX);
4     HRTIM_CompareCfgTypeDef cmp = {0};
5     uint32_t cmp1Ticks = Duty_ToTicks(duty);
6     uint32_t cmp3Ticks = cmp1Ticks + HRTIM_DT_TICKS;
7     if (cmp3Ticks >= HRTIM_PERIOD_TICKS)
8         cmp3Ticks = HRTIM_PERIOD_TICKS - 1U;
9     cmp.CompareValue = cmp3Ticks;
10    HAL_HRTIM_WaveformCompareConfig(&hhrtim1,
11        HRTIM_TIMERINDEX_TIMER_A, HRTIM_COMPAREUNIT_3, &cmp);
12    cmp.CompareValue = HRTIM_CMP2_TICKS;
13    cmp.AutoDelayedMode = HRTIM_AUTODELAYEDMODE_REGULAR;
14    cmp.AutoDelayedTimeout = 0x0000;
15    HAL_HRTIM_WaveformCompareConfig(&hhrtim1,

```

```
16     HRTIM_TIMERINDEX_TIMER_A, HRTIM_COMPAREUNIT_2, &cmp);
17     HAL_HRTIM_WaveformOutputStop(&hhrtim1, HRTIM_OUTPUT_TA1);
18     HAL_HRTIM_WaveformOutputStart(&hhrtim1, HRTIM_OUTPUT_TA2);
19 }
```

Listing 8: Motor_SetDirB — Direction B Output Drive (main.c)

```
1 static float ADC_ReadDutyReference(void)
2 {
3     HAL_ADC_Start(&hadc1);
4     if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
5     {
6         uint32_t raw = HAL_ADC_GetValue(&hadc1);
7         float duty = (float)raw / ADC_FULL_SCALE;
8         HAL_ADC_Stop(&hadc1);
9         return CLAMP_F(duty, DUTY_MIN, DUTY_MAX);
10    }
11    HAL_ADC_Stop(&hadc1);
12    return 0.0f;
13 }
```

Listing 9: ADC_ReadDutyReference — Software-Triggered ADC Read (main.c)

D Hardware Documentation

Refer to the official NUCLEO-G474RE schematic (UM2505) for complete hardware specifications, connector pinouts, and the on-board pull-down resistor arrangement on the B1 user button (PC13).

Author

Anmol Govindarajapuram Krishnan

GitHub: <https://github.com/Anmol-G-K>

Email: cb.en.u4eee23103@cb.students.amrita.edu

Acknowledgments

We acknowledge the following for their contributions and support:

- **STMicroelectronics** for providing comprehensive microcontroller documentation and development tools
- **MathWorks** for excellent MATLAB and Embedded Coder resources
- The embedded systems and power electronics community for innovation and knowledge sharing
- Contributors and reviewers providing feedback

This project is licensed under the MIT License.

For updates and additional resources:

<https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>